


# Misión Órbita: Avance 1 – Preparación de la misión

Josue Monge Miranda

Escuela de Ingeniería del Software, Universidad Cenfotec

SOFT-01 Principios de programación 1, sección SCV3

Verónica Mora Lezcano

14 de octubre de 2026

Repositorio: https://github.com/jmonge12/Misi-n-rbita---Avance-1

## Misión Órbita: Avance 1 – Preparación de la misión

Este documento presenta el primer avance del proyecto Misión Órbita, un simulador básico de exploración espacial hecho en Python. En esta etapa se resolvieron cuatro algoritmos independientes que preparan la misión: el cálculo de recursos, la verificación de combustible, la verificación del soporte para la tripulación y la autorización de lanzamiento (Mora Lezcano, 2026).

Cada ejercicio se analizó antes de programarlo, así que se presenta su tabla de variables, su diagrama de flujo, el código fuente y los casos de prueba con los resultados esperados y obtenidos. Todo sigue el esquema entrada, procesamiento, decisión y salida, y respeta las restricciones técnicas de la consigna: solo consola, sin ciclos, sin colecciones, sin funciones propias y sin condicionales anidados ni elif.

## Ejercicio 1: Cálculo de recursos necesarios para una misión

Este programa pide el nombre de la misión, la cantidad de tripulantes y los días que tarda el viaje hasta el destino. Con esos datos calcula el combustible de ida, de regreso y de reserva, el combustible total, y los requerimientos de oxígeno, energía y provisiones. Es un flujo completamente secuencial, sin decisiones.

La consigna indica que el oxígeno, la energía y las provisiones se consumen durante el viaje de ida y regreso, por eso esas fórmulas se multiplican por la constante TRAMOS_VIAJE. Con esto, el resultado del ejemplo de la consigna (65 unidades de oxígeno para 3 tripulantes durante 5 días) coincide con la salida del programa.

### Tabla de variables

**Tabla 1**
*Variables y constantes del ejercicio 1*

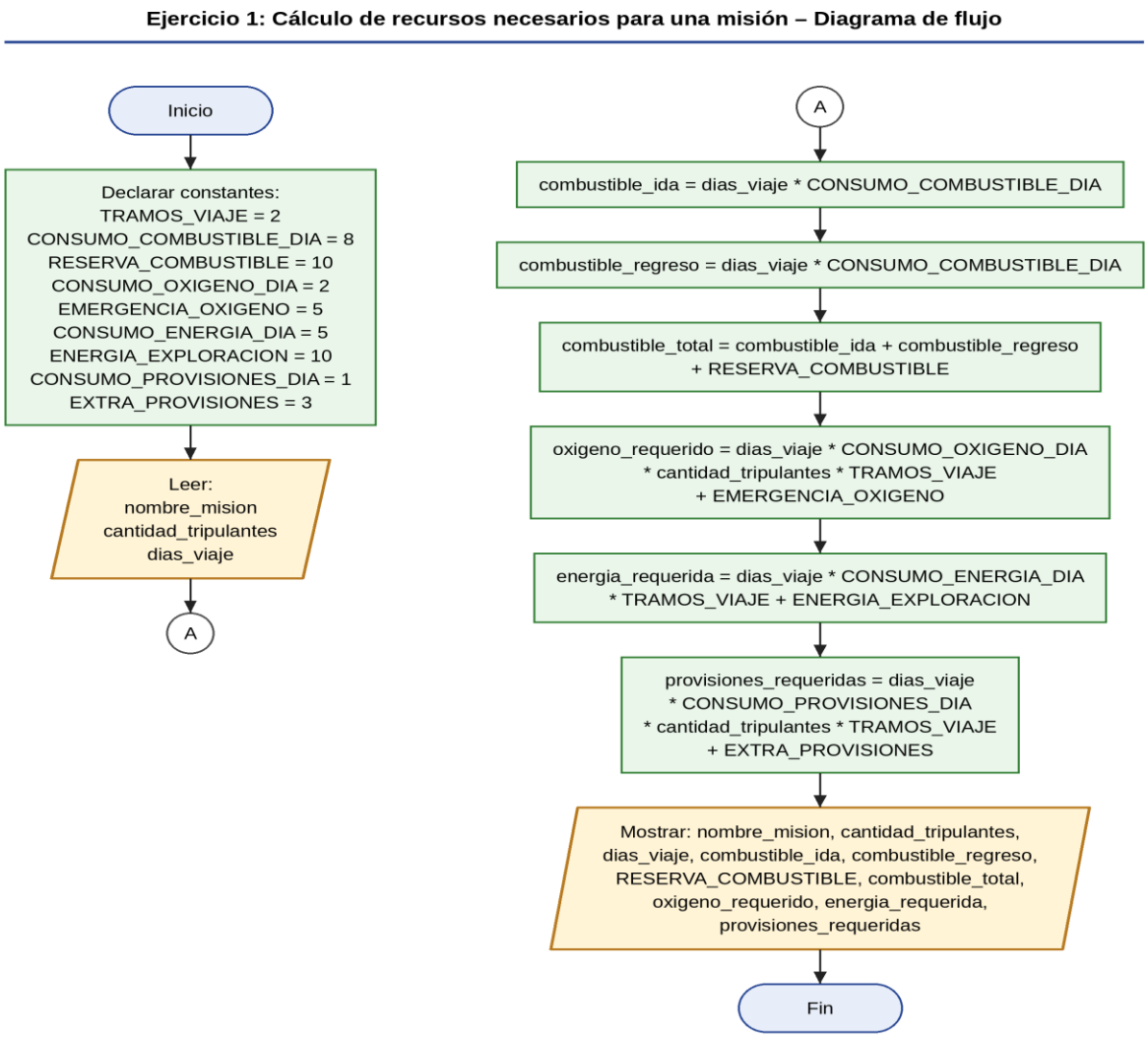
| Nombre | Tipo de dato | Clasificación | Descripción | Valor inicial / restricción |
|---|---|---|---|---|
| TRAMOS_VIAJE | int | Constante | Cantidad de tramos del viaje (ida y regreso). | 2 |
| CONSUMO_COMBUSTIBLE_DIA | int | Constante | Combustible que se consume por día de viaje. | 8 |
| RESERVA_COMBUSTIBLE | int | Constante | Combustible adicional para emergencias. | 10 |
| CONSUMO_OXIGENO_DIA | int | Constante | Oxígeno que consume cada tripulante por día. | 2 |
| EMERGENCIA_OXIGENO | int | Constante | Oxígeno adicional de emergencia. | 5 |
| CONSUMO_ENERGIA_DIA | int | Constante | Energía que consume la nave por día. | 5 |
| ENERGIA_EXPLORACION | int | Constante | Energía reservada para la exploración. | 10 |
| CONSUMO_PROVISIONES_DIA | int | Constante | Provisiones que consume cada tripulante por día. | 1 |
| EXTRA_PROVISIONES | int | Constante | Provisiones adicionales. | 3 |
| nombre_mision | str | Entrada | Nombre de la misión. | Texto digitado |
| cantidad_tripulantes | int | Entrada | Número de tripulantes de la nave. | Entero positivo |
| dias_viaje | int | Entrada | Días estimados para llegar al destino. | Entero positivo |
| combustible_ida | int | Salida | Combustible necesario para llegar al destino. | Calculado |
| combustible_regreso | int | Salida | Combustible necesario para el regreso. | Calculado |

| Nombre | Tipo de dato | Clasificación | Descripción | Valor inicial / restricción |
|---|---|---|---|---|
| `combustible_total` | int | Salida | Ida + regreso + reserva. | Calculado |
| `oxigeno_requerido` | int | Salida | Oxígeno total de la misión. | Calculado |
| `energia_requerida` | int | Salida | Energía total de la misión. | Calculado |
| `provisiones_requeridas` | int | Salida | Provisiones totales de la misión. | Calculado |

## Diagrama de flujo

**Figura 1**

*Diagrama de flujo del ejercicio 1*



## Código fuente

**Figura 2**
*Código del archivo ejercicio1_recursos.py*

```
# ================================================================
# Archivo: ejercicio1_recursos.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Josue Monge Miranda
# Fecha: 14/10/2026    Versión: 1.1
# Descripción: Calcula los recursos necesarios para una misión.
# ================================================================

# --- Declaración de constantes ---
# Van al inicio para no dejar números mágicos dentro de las fórmulas.
TRAMOS_VIAJE = 2                          # ida y regreso
CONSUMO_COMBUSTIBLE_DIA = 8
RESERVA_COMBUSTIBLE = 10
CONSUMO_OXIGENO_DIA = 2
EMERGENCIA_OXIGENO = 5
CONSUMO_ENERGIA_DIA = 5
ENERGIA_EXPLORACION = 10
CONSUMO_PROVISIONES_DIA = 1
EXTRA_PROVISIONES = 3

# --- Entrada de datos ---
nombre_mision = input("Nombre de la misión: ")
cantidad_tripulantes = int(input("Cantidad de tripulantes: "))
dias_viaje = int(input("Días estimados para llegar al destino: "))

# --- Procesamiento ---
# El regreso dura lo mismo que la ida, por eso el consumo de oxígeno,
# energía y provisiones se multiplica por TRAMOS_VIAJE.
combustible_ida = dias_viaje * CONSUMO_COMBUSTIBLE_DIA
combustible_regreso = dias_viaje * CONSUMO_COMBUSTIBLE_DIA
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

oxigeno_requerido = (dias_viaje * CONSUMO_OXIGENO_DIA
                     * cantidad_tripulantes * TRAMOS_VIAJE
                     + EMERGENCIA_OXIGENO)
energia_requerida = (dias_viaje * CONSUMO_ENERGIA_DIA * TRAMOS_VIAJE
                     + ENERGIA_EXPLORACION)
provisiones_requeridas = (dias_viaje * CONSUMO_PROVISIONES_DIA
                          * cantidad_tripulantes * TRAMOS_VIAJE
                          + EXTRA_PROVISIONES)

# --- Salida de resultados ---
print()
print("Misión:", nombre_mision)
print("Tripulantes:", cantidad_tripulantes)
print("Duración estimada hasta el destino:", dias_viaje, "días")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de combustible:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxígeno requerido:", oxigeno_requerido, "unidades")
print("Energía requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")
```

## Casos de prueba

**Tabla 2**
*Casos de prueba del ejercicio 1*

| ID | Ejercicio | Escenario | Datos de entrada | Resultado esperado | Resultado obtenido | Estado |
|---|---|---|---|---|---|---|
| CP-01 | Ej. 1 | Misión del ejemplo de la consigna | Explorador I<br>3 tripulantes<br>5 días | Ida 40, regreso 40, reserva 10<br>Total 90<br>Oxígeno 65, energía 60<br>Provisiones 33 | Ida 40, regreso 40, reserva 10<br>Total 90<br>Oxígeno 65, energía 60<br>Provisiones 33 | Aprobado |
| CP-02 | Ej. 1 | Misión corta con un solo tripulante | Apolo X<br>1 tripulante<br>2 días | Ida 16, regreso 16, reserva 10<br>Total 16 + 16 + 10 = 42<br>Oxígeno 2*2*1*2+5 = 13<br>Energía 2*5*2+10 = 30<br>Provisiones 2*1*1*2+3 = 7 | Ida 16, regreso 16, reserva 10<br>Total 42<br>Oxígeno 13, energía 30<br>Provisiones 7 | Aprobado |
| CP-03 | Ej. 1 | Misión larga con tripulación grande | Titan I<br>5 tripulantes<br>10 días | Ida 80, regreso 80, reserva 10<br>Total 80 + 80 + 10 = 170<br>Oxígeno 10*2*5*2+5 = 205<br>Energía 10*5*2+10 = 110<br>Provisiones 10*1*5*2+3 = 103 | Ida 80, regreso 80, reserva 10<br>Total 170<br>Oxígeno 205, energía 110<br>Provisiones 103 | Aprobado |

**Figura 3**
*Salida de consola de los casos CP-01 a CP-03*

```
--- CP-01 ---
Nombre de la misión: Explorador I
Cantidad de tripulantes: 3
Días estimados para llegar al destino: 5

Misión: Explorador I
Tripulantes: 3
Duración estimada hasta el destino: 5 días
Combustible para llegar: 40 unidades
Combustible para regresar: 40 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 90 unidades
Oxígeno requerido: 65 unidades
Energía requerida: 60 unidades
Provisiones requeridas: 33 unidades

--- CP-02 ---
Nombre de la misión: Apolo X
Cantidad de tripulantes: 1
Días estimados para llegar al destino: 2

Misión: Apolo X
Tripulantes: 1
Duración estimada hasta el destino: 2 días
Combustible para llegar: 16 unidades
Combustible para regresar: 16 unidades
Reserva de combustible: 10 unidades
```

```
Combustible total requerido: 42 unidades
Oxígeno requerido: 13 unidades
Energía requerida: 30 unidades
Provisiones requeridas: 7 unidades


--- CP-03 ---
Nombre de la misión: Titan I
Cantidad de tripulantes: 5
Días estimados para llegar al destino: 10


Misión: Titan I
Tripulantes: 5
Duración estimada hasta el destino: 10 días
Combustible para llegar: 80 unidades
Combustible para regresar: 80 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 170 unidades
Oxígeno requerido: 205 unidades
Energía requerida: 110 unidades
Provisiones requeridas: 103 unidades
```

## Ejercicio 2: Verificación de combustible

Este programa compara el combustible disponible con el total que necesita la misión (ida, regreso y una reserva de 10 unidades) y calcula el combustible adicional. Siempre muestra las seis salidas y solo imprime la advertencia cuando el adicional es menor que 10, para lo cual se usa un condicional simple (if) sin else.

Se eligió el operador < y no <= porque con exactamente 10 unidades de margen todavía no corresponde advertir, y eso se comprueba en el caso límite CP-06. Si el combustible no alcanza ni para el total, el adicional queda negativo y también se muestra la advertencia, porque la consigna no define otro mensaje para esa situación (CP-07).

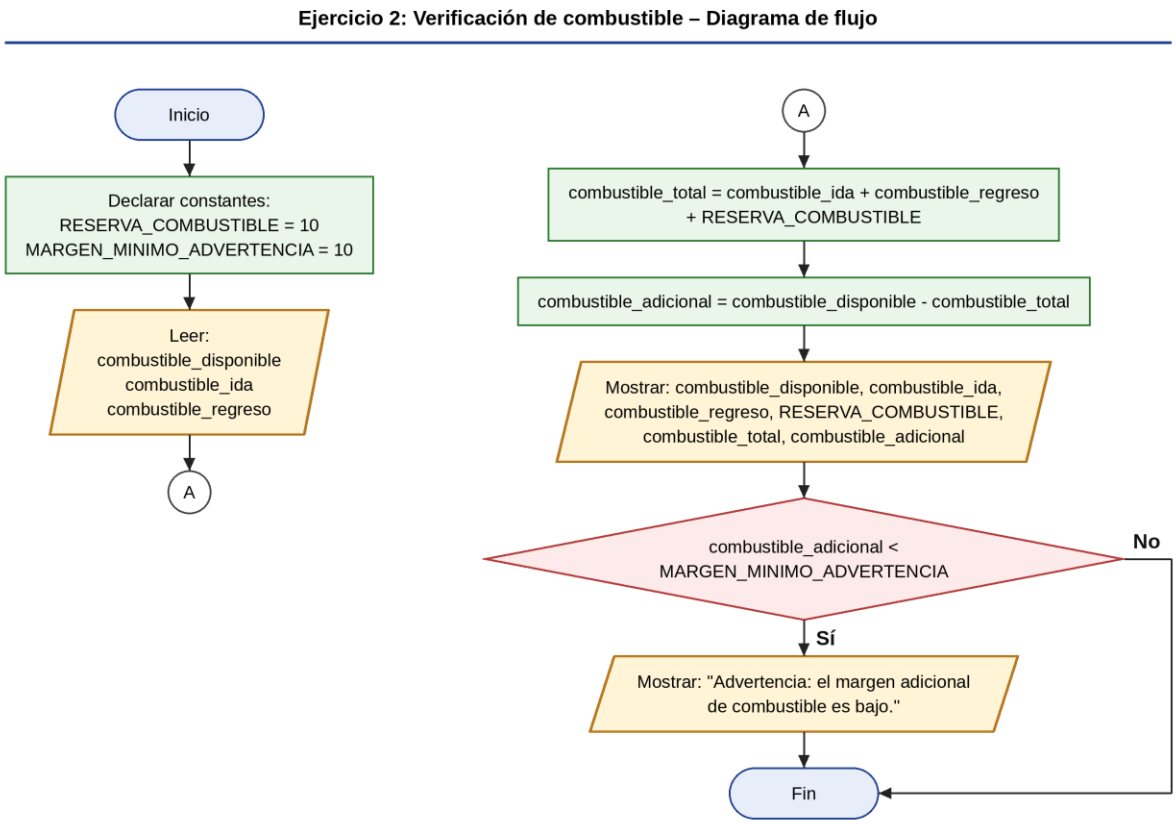
### Tabla de variables

**Tabla 3**
*Variables y constantes del ejercicio 2*

| Nombre | Tipo de dato | Clasificación | Descripción | Valor inicial / restricción |
|---|---|---|---|---|
| `RESERVA_COMBUSTIBLE` | int | Constante | Reserva de emergencia del combustible. | 10 |
| `MARGEN_MINIMO_ADVERTENCIA` | int | Constante | Margen adicional mínimo para no mostrar advertencia. | 10 |
| `combustible_disponible` | int | Entrada | Combustible que tiene la nave. | Entero positivo |
| `combustible_ida` | int | Entrada | Combustible requerido para la ida. | Entero positivo |
| `combustible_regreso` | int | Entrada | Combustible requerido para el regreso. | Entero positivo |
| `combustible_total` | int | Salida | Ida + regreso + reserva. | Calculado |
| `combustible_adicional` | int | Salida | Disponible menos el total requerido. | Calculado (puede ser negativo) |

## Diagrama de flujo

**Figura 4**
*Diagrama de flujo del ejercicio 2*




## Código fuente

**Figura 5**
*Código del archivo ejercicio2_combustible.py*

```
# ====================================================================
# Archivo: ejercicio2_combustible.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Josue Monge Miranda
# Fecha: 14/10/2026    Versión: 1.1
# Descripción: Verifica si el combustible disponible cubre la misión
#              y advierte cuando el margen adicional es bajo.
# ====================================================================

# --- Declaración de constantes ---
RESERVA_COMBUSTIBLE = 10
MARGEN_MINIMO_ADVERTENCIA = 10

# --- Entrada de datos ---
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para la ida: "))
```

```
combustible_regreso = int(input("Combustible requerido para el regreso: "))

# --- Procesamiento ---
# El total incluye el regreso y la reserva, no solo la ida.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
combustible_adicional = combustible_disponible - combustible_total

# --- Salida de resultados (siempre se muestran las seis) ---
print()
print("Combustible disponible:", combustible_disponible, "unidades")
print("Combustible requerido para llegar:", combustible_ida, "unidades")
print("Combustible requerido para regresar:", combustible_regreso, "unidades")
print("Reserva de emergencia:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Combustible adicional disponible:", combustible_adicional, "unidades")

# --- Decisión (condicional simple) ---
# Se usa < y no <= porque con exactamente 10 unidades el margen es aceptable.
if combustible_adicional < MARGEN_MINIMO_ADVERTENCIA:
    print("Advertencia: el margen adicional de combustible es bajo.")
```

## Casos de prueba

**Tabla 4**

*Casos de prueba del ejercicio 2*

| ID | Ejercicio | Escenario | Datos de entrada | Resultado esperado | Resultado obtenido | Estado |
|---|---|---|---|---|---|---|
| CP-04 | Ej. 2 | Margen adicional holgado (mayor que 10) | Disponible = 110<br>Ida = 40<br>Regreso = 40 | Total = 90<br>Adicional = 20<br>No muestra advertencia | Total 90<br>Adicional 20<br>Sin advertencia | Aprobado |
| CP-05 | Ej. 2 | Margen adicional bajo (menor que 10) | Disponible = 95<br>Ida = 40<br>Regreso = 40 | Total = 90<br>Adicional = 5<br>Muestra advertencia | Total 90<br>Adicional 5<br>Muestra advertencia | Aprobado |
| CP-06 | Ej. 2 | Caso límite: adicional exactamente 10 | Disponible = 100<br>Ida = 40<br>Regreso = 40 | Total = 90<br>Adicional = 10<br>No muestra advertencia (10 no es menor que 10) | Total 90<br>Adicional 10<br>Sin advertencia | Aprobado |
| CP-07 | Ej. 2 | Combustible solo para llegar | Disponible = 45<br>Ida = 40<br>Regreso = 40 | Total = 90<br>Adicional = 45 - 90 = -45<br>Muestra advertencia | Total 90<br>Adicional -45<br>Muestra advertencia | Aprobado |

**Figura 6**

*Salida de consola de los casos CP-04 a CP-07*

```
--- CP-04 ---
Combustible disponible: 110
Combustible requerido para la ida: 40
Combustible requerido para el regreso: 40

Combustible disponible: 110 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
```

```
Combustible adicional disponible: 20 unidades

--- CP-05 ---
Combustible disponible: 95
Combustible requerido para la ida: 40
Combustible requerido para el regreso: 40

Combustible disponible: 95 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: 5 unidades
Advertencia: el margen adicional de combustible es bajo.

--- CP-06 ---
Combustible disponible: 100
Combustible requerido para la ida: 40
Combustible requerido para el regreso: 40

Combustible disponible: 100 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: 10 unidades

--- CP-07 ---
Combustible disponible: 45
Combustible requerido para la ida: 40
Combustible requerido para el regreso: 40

Combustible disponible: 45 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: -45 unidades
Advertencia: el margen adicional de combustible es bajo.
```

## Ejercicio 3: Verificación del soporte para la tripulación

Aquí se revisa si la nave tiene oxígeno y provisiones suficientes mediante un if/else con una condición compuesta. Se usa and porque ambos recursos tienen que cumplirse al mismo tiempo, y si falla cualquiera de los dos se muestra el mismo mensaje negativo, ya que la consigna pide no indicar cuál recurso presenta el problema.

Las comparaciones usan >= porque tener justo lo requerido es suficiente, y por eso el caso CP-09 usa valores exactamente iguales a los mínimos.

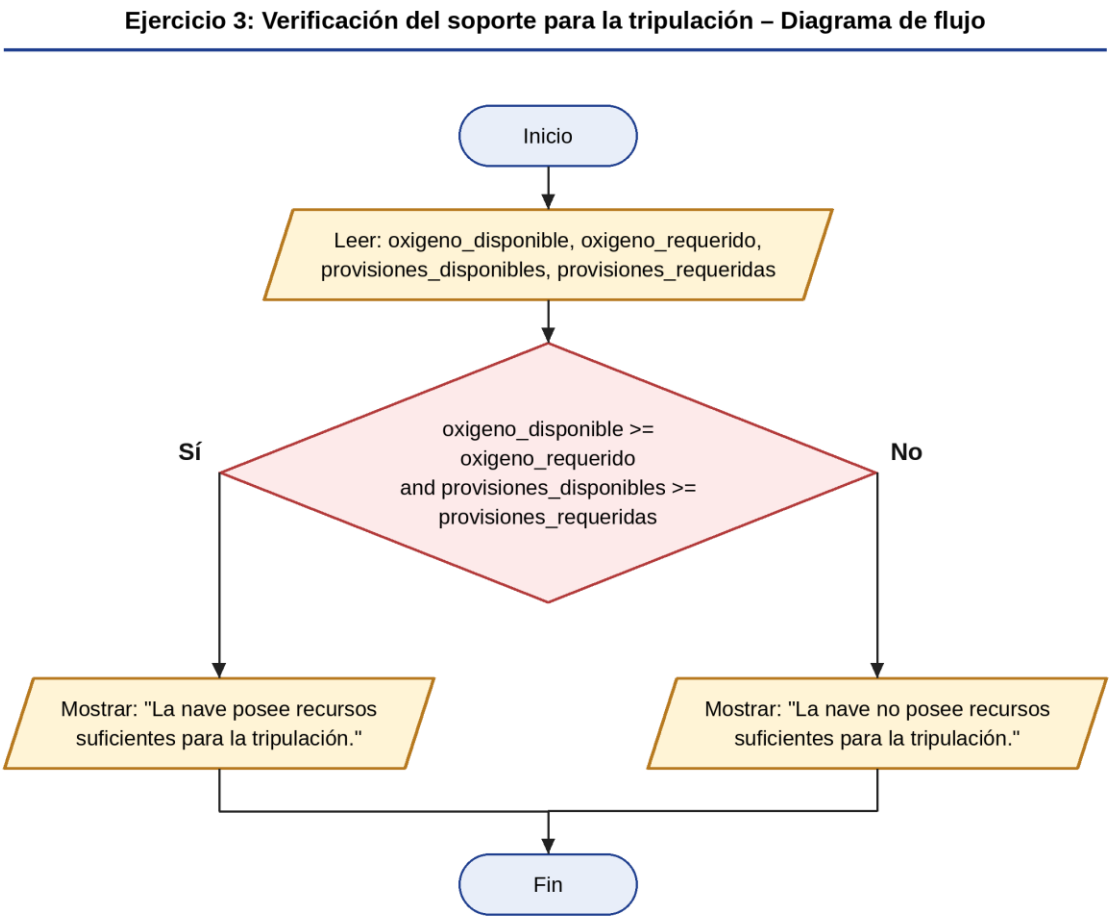
### Tabla de variables

**Tabla 5**
*Variables y constantes del ejercicio 3*

| Nombre | Tipo de dato | Clasificación | Descripción | Valor inicial / restricción |
|---|---|---|---|---|
| `oxigeno_disponible` | int | Entrada | Oxígeno que tiene la nave. | Entero positivo |
| `oxigeno_requerido` | int | Entrada | Oxígeno que necesita la misión. | Entero positivo |
| `provisiones_disponibles` | int | Entrada | Provisiones que tiene la nave. | Entero positivo |
| `provisiones_requeridas` | int | Entrada | Provisiones que necesita la misión. | Entero positivo |

## Diagrama de flujo

**Figura 7**
*Diagrama de flujo del ejercicio 3*




## Código fuente

**Figura 8**
*Código del archivo ejercicio3_soporte_tripulacion.py*

```
# ====================================================================
# Archivo: ejercicio3_soporte_tripulacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Josue Monge Miranda
# Fecha: 14/10/2026    Versión: 1.1
# Descripción: Verifica si la nave tiene oxígeno y provisiones
#              suficientes para la tripulación.
# ====================================================================

# --- Entrada de datos ---
oxigeno_disponible = int(input("Oxígeno disponible: "))
oxigeno_requerido = int(input("Oxígeno requerido: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
```

```
provisiones_requeridas = int(input("Provisiones requeridas: "))

# --- Decisión (condicional doble) ---
# Se usa "and" porque la misión solo es viable si se cumplen ambos recursos.
# No se indica cuál recurso falla, tal como lo pide la consigna.
if (oxigeno_disponible >= oxigeno_requerido
        and provisiones_disponibles >= provisiones_requeridas):
    print("La nave posee recursos suficientes para la tripulación.")
else:
    print("La nave no posee recursos suficientes para la tripulación.")
```

## Casos de prueba

**Tabla 6**

*Casos de prueba del ejercicio 3*

| ID | Ejercicio | Escenario | Datos de entrada | Resultado esperado | Resultado obtenido | Estado |
|---|---|---|---|---|---|---|
| CP-08 | Ej. 3 | Recursos suficientes | Oxígeno: 70 disp., 65 req. Provisiones: 40 disp., 33 req. | La nave posee recursos suficientes para la tripulación. | La nave posee recursos suficientes para la tripulación. | Aprobado |
| CP-09 | Ej. 3 | Caso límite: valores exactamente iguales | Oxígeno: 65 disp., 65 req. Provisiones: 33 disp., 33 req. | La nave posee recursos suficientes para la tripulación. | La nave posee recursos suficientes para la tripulación. | Aprobado |
| CP-10 | Ej. 3 | Provisiones insuficientes | Oxígeno: 70 disp., 65 req. Provisiones: 20 disp., 33 req. | La nave no posee recursos suficientes para la tripulación. | La nave no posee recursos suficientes para la tripulación. | Aprobado |
| CP-11 | Ej. 3 | Oxígeno insuficiente | Oxígeno: 50 disp., 65 req. Provisiones: 40 disp., 33 req. | La nave no posee recursos suficientes para la tripulación. | La nave no posee recursos suficientes para la tripulación. | Aprobado |

**Figura 9**

*Salida de consola de los casos CP-08 a CP-11*

```
--- CP-08 ---
Oxígeno disponible: 70
Oxígeno requerido: 65
Provisiones disponibles: 40
Provisiones requeridas: 33
La nave posee recursos suficientes para la tripulación.

--- CP-09 ---
Oxígeno disponible: 65
Oxígeno requerido: 65
Provisiones disponibles: 33
Provisiones requeridas: 33
La nave posee recursos suficientes para la tripulación.

--- CP-10 ---
Oxígeno disponible: 70
Oxígeno requerido: 65
Provisiones disponibles: 20
Provisiones requeridas: 33
```

```
La nave no posee recursos suficientes para la tripulación.

--- CP-11 ---
Oxígeno disponible: 50
Oxígeno requerido: 65
Provisiones disponibles: 40
Provisiones requeridas: 33
La nave no posee recursos suficientes para la tripulación.
```

## Ejercicio 4: Autorización inicial de lanzamiento

Este programa decide si se autoriza el lanzamiento. Primero calcula el combustible total (ida, regreso y reserva) y después evalúa los cuatro recursos en un único if/else, sin condicionales anidados.

El combustible disponible se compara contra el total y no solo contra el de ida, de modo que una nave que alcanza únicamente para llegar al destino queda rechazada (CP-14). El caso CP-15 comprueba que tampoco se autorice cuando alcanza para ida y regreso pero falta la reserva.

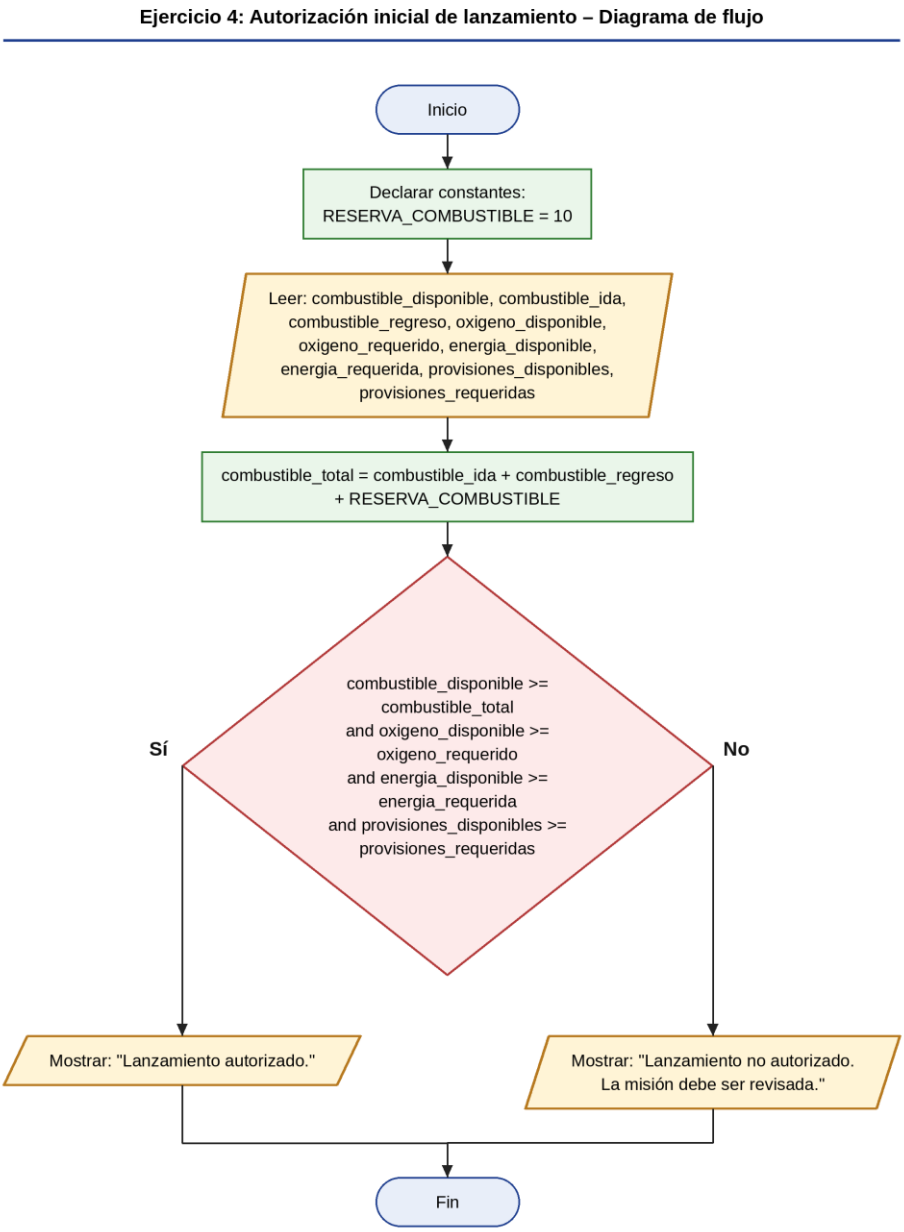
### Tabla de variables

**Tabla 7**
*Variables y constantes del ejercicio 4*

| Nombre | Tipo de dato | Clasificación | Descripción | Valor inicial / restricción |
|---|---|---|---|---|
| `RESERVA_COMBUSTIBLE` | int | Constante | Reserva de emergencia del combustible. | 10 |
| `combustible_disponible` | int | Entrada | Combustible que tiene la nave. | Entero positivo |
| `combustible_ida` | int | Entrada | Combustible requerido para llegar al destino. | Entero positivo |
| `combustible_regreso` | int | Entrada | Combustible requerido para regresar. | Entero positivo |
| `oxigeno_disponible` | int | Entrada | Oxígeno que tiene la nave. | Entero positivo |
| `oxigeno_requerido` | int | Entrada | Oxígeno que necesita la misión. | Entero positivo |
| `energia_disponible` | int | Entrada | Energía que tiene la nave. | Entero positivo |
| `energia_requerida` | int | Entrada | Energía que necesita la misión. | Entero positivo |
| `provisiones_disponibles` | int | Entrada | Provisiones que tiene la nave. | Entero positivo |
| `provisiones_requeridas` | int | Entrada | Provisiones que necesita la misión. | Entero positivo |
| `combustible_total` | int | Intermedia | Ida + regreso + reserva; se usa en la decisión. | Calculado |

## Diagrama de flujo

**Figura 10**
*Diagrama de flujo del ejercicio 4*




## Código fuente

**Figura 11**
*Código del archivo ejercicio4_autorizacion.py*

```
# ======================================================================
```

```
# Archivo: ejercicio4_autorizacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Josue Monge Miranda
# Fecha: 14/10/2026    Versión: 1.1
# Descripción: Autoriza o rechaza el lanzamiento según combustible,
#              oxígeno, energía y provisiones.
# ========================================================================

# --- Declaración de constantes ---
RESERVA_COMBUSTIBLE = 10

# --- Entrada de datos ---
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para llegar al destino: "))
combustible_regreso = int(input("Combustible requerido para regresar: "))
oxigeno_disponible = int(input("Oxígeno disponible: "))
oxigeno_requerido = int(input("Oxígeno requerido: "))
energia_disponible = int(input("Energía disponible: "))
energia_requerida = int(input("Energía requerida: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
provisiones_requeridas = int(input("Provisiones requeridas: "))

# --- Procesamiento ---
# El combustible se compara contra el total (ida + regreso + reserva),
# así no se autoriza una misión que solo alcanza para llegar.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# --- Decisión (condicional doble, sin anidar) ---
if (combustible_disponible >= combustible_total
        and oxigeno_disponible >= oxigeno_requerido
        and energia_disponible >= energia_requerida
        and provisiones_disponibles >= provisiones_requeridas):
    print("Lanzamiento autorizado.")
else:
    print("Lanzamiento no autorizado. La misión debe ser revisada.")
```

## Casos de prueba

**Tabla 8**

*Casos de prueba del ejercicio 4*

| ID | Ejercicio | Escenario | Datos de entrada | Resultado esperado | Resultado obtenido | Estado |
|---|---|---|---|---|---|---|
| CP-12 | Ej. 4 | Todos los recursos suficientes | Combustible: 100 disp. (ida 40, regreso 40)<br>Oxígeno: 70/65<br>Energía: 70/60<br>Provisiones: 40/33 | Lanzamiento autorizado. | Lanzamiento autorizado. | Aprobado |
| CP-13 | Ej. 4 | Caso límite: todo exactamente igual al mínimo | Combustible: 90 disp. (ida 40, regreso 40)<br>Oxígeno: 65/65<br>Energía: 60/60<br>Provisiones: 33/33 | Lanzamiento autorizado. | Lanzamiento autorizado. | Aprobado |

| ID | Ejercicio | Escenario | Datos de entrada | Resultado esperado | Resultado obtenido | Estado |
|---|---|---|---|---|---|---|
| CP-14 | Ej. 4 | Combustible solo para llegar | Combustible: 45 disp. (ida 40, regreso 40)<br>Oxígeno: 70/65<br>Energía: 70/60<br>Provisiones: 40/33 | Lanzamiento no autorizado. La misión debe ser revisada. | Lanzamiento no autorizado. La misión debe ser revisada. | Aprobado |
| CP-15 | Ej. 4 | Combustible para ida y regreso, pero sin la reserva | Combustible: 80 disp. (ida 40, regreso 40)<br>Oxígeno: 70/65<br>Energía: 70/60<br>Provisiones: 40/33 | Lanzamiento no autorizado. La misión debe ser revisada. | Lanzamiento no autorizado. La misión debe ser revisada. | Aprobado |
| CP-16 | Ej. 4 | Energía insuficiente | Combustible: 100 disp. (ida 40, regreso 40)<br>Oxígeno: 70/65<br>Energía: 50/60<br>Provisiones: 40/33 | Lanzamiento no autorizado. La misión debe ser revisada. | Lanzamiento no autorizado. La misión debe ser revisada. | Aprobado |

**Figura 12**

*Salida de consola de los casos CP-12 a CP-16*

```
--- CP-12 ---
Combustible disponible: 100
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 70
Oxígeno requerido: 65
Energía disponible: 70
Energía requerida: 60
Provisiones disponibles: 40
Provisiones requeridas: 33
Lanzamiento autorizado.

--- CP-13 ---
Combustible disponible: 90
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 65
Oxígeno requerido: 65
Energía disponible: 60
Energía requerida: 60
Provisiones disponibles: 33
Provisiones requeridas: 33
Lanzamiento autorizado.

--- CP-14 ---
Combustible disponible: 45
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 70
Oxígeno requerido: 65
Energía disponible: 70
Energía requerida: 60
Provisiones disponibles: 40
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

--- CP-15 ---
```

```
Combustible disponible: 80
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 70
Oxígeno requerido: 65
Energía disponible: 70
Energía requerida: 60
Provisiones disponibles: 40
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

--- CP-16 ---
Combustible disponible: 100
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 70
Oxígeno requerido: 65
Energía disponible: 50
Energía requerida: 60
Provisiones disponibles: 40
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.
```

## Repositorio Git

El código y la documentación están en el repositorio de GitHub: https://github.com/jmonge12/Misi-n-rbita---Avance-1. Sigue la estructura sugerida en la consigna y el avance se fue registrando con commits de mensajes descriptivos. En la Figura 13 se muestra el contenido del README.md.

**Figura 13**
*Contenido del archivo README.md del repositorio*

```
# Misión Órbita - Avance 1

## Descripción
Simulador básico de exploración espacial desarrollado en Python para el curso
Principios de Programación 1 (SOFT-01, sección SCV3) de la Universidad CENFOTEC.
En este avance se implementan cuatro algoritmos independientes que representan
la preparación de la misión.

## Integrantes
- Josue Monge Miranda

## Estructura del repositorio
mision-orbita/
├── README.md
├── avance1/
│   ├── ejercicio1_recursos.py
│   ├── ejercicio2_combustible.py
│   ├── ejercicio3_soporte_tripulacion.py
│   └── ejercicio4_autorizacion.py
└── documentacion/
    └── avance1.pdf
```

```
## Instrucciones de ejecución
Cada programa es independiente y se ejecuta desde la consola con Python 3:

    python avance1/ejercicio1_recursos.py
    python avance1/ejercicio2_combustible.py
    python avance1/ejercicio3_soporte_tripulacion.py
    python avance1/ejercicio4_autorizacion.py

Todos los datos se ingresan por consola como números enteros positivos.

## Convenciones y estilos
- Variables en snake_case, con nombres descriptivos y en minúscula.
- Constantes en MAYÚSCULAS con guion bajo, declaradas al inicio del archivo.
- Sin números mágicos dentro de las fórmulas.
- Indentación de 4 espacios y espacios alrededor de los operadores.
- Encabezado en cada archivo (archivo, curso, integrantes, fecha, versión y descripción).
- Un comentario antes de cada bloque lógico: constantes, entrada, procesamiento,
  decisión y salida.
- Mensajes de consola con sus unidades.
- Commits con mensajes que describen el cambio realizado.
```

## Referencias

Mora Lezcano, V. (2026). *Consigna del primer avance del proyecto: Preparación de la misión* [Documento de curso]. Escuela de Ingeniería del Software, Universidad Cenfotec.